Using SAT features to guide Tableau provers

(possibly for AITP, hopefully for my first-year report)

Introduction and Research

Rawson and Reger introduce SATCoP [5], a tableau prover based on LeanCoP [4] that runs a separate SAT solver along fed with grounded atoms from the tableau prover. If the SAT solver reaches an unsatisfiable state, then we can prove that the entire tree is unsatisfiable.

This approach is sound, but incomplete. There are many problems that cannot be directly proven by this combination.

Novel Idea

Besides saying whether the current prover is UNSAT, CaDiCaL [1] gives us extra information.

`probe_conflicts`
: For every SAT literal a, will $\neg a$ cause UNSAT?

`forced_true`
: Is this candidate subgoal forced true by the SAT solver?

`evsids`
: Sum of EVSIDS scores for all grounded atoms of a certain formula.

`subgoal_count`
: Number of remaining subgoals introduced by the extension (lower is better). This is similar to one of the heuristics used by Letz and Stenz [3].

Implementation

I create a strategy, `SATSolveGuided`, which chooses the start, extension, or reduction with the highest one of these properties. It prefers starts first, reductions later, and extensions only of none of these two are presents.

However, experimentation showed that calculating these properties for all leafs is slow. For this reason I experiment with 6 different strategies.

SATSolveGuided
: `(probe_conflicts, forced_true, evsids, -subgoal_count)`

SATSolveGuidedProbeConflicts
: `(probe_conflicts)`

SATSolveGuidedForcedTrue
: `(forced_true)`

SATSolveGuidedEVSIDS
: `(evsids)`

SATSolveGuidedShort
: `(-subgoal_count)`

SATSolveGuidedFast
: `(forced_true, evsids, -subgoal_count)`

I use the existing connection prover implementation as a base for this code [6]. This allows me to use the existing code for hard things like Prolog parsing, and also to use PyCoP as a control group.

Results

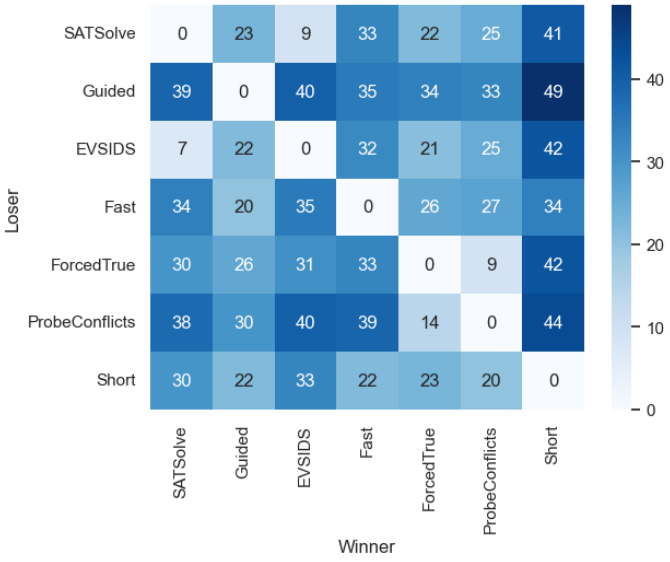


Figure 1: Winners and losers of different SAT policies in a subset of 1458 problems [2]. I expect these percentages to be similar in the whole set.

Experiments were run on the University of Cambridge CSD3 Ice Lake cluster, using one node per experiment with 76 physical CPU cores and 256 GiB RAM.

Problem	Strategy	Solved	Steps	Time (ms)
KRS255+1.p	SATSolve	false	12 659	90 003
	SATSolveGuided	false	9104	90 002
	SATSolveGuidedProbeConflicts	false	10 533	90 002
	SATSolveGuidedForcedTrue	true	7423	40 270
	SATSolveGuidedEVSIDS	true	6516	39 185
	SATSolveGuidedShort	false	13 201	90 007
	SATSolveGuidedFast	false	12 547	90 011
GEO214+2.p	SATSolve	false	24 308	90 001
	SATSolveGuided	true	4831	4622
	SATSolveGuidedProbeConflicts	false	90 966	90 000
	SATSolveGuidedForcedTrue	false	93 502	90 000
	SATSolveGuidedEVSIDS	false	24 030	90 013
	SATSolveGuidedShort	true	15 944	34 071
	SATSolveGuidedFast	true	4440	3816
SYN578-1.p	SATSolve	false	–	–
	SATSolveGuided	true	21 388	103 259
	SATSolveGuidedProbeConflicts	true	11 347	33 783
	SATSolveGuidedForcedTrue	true	11 994	32 949
	SATSolveGuidedEVSIDS	false	–	–
	SATSolveGuidedShort	true	25 291	100 802
	SATSolveGuidedFast	true	11 083	22 460
SYN583-1.p	SATSolve	false	–	–
	SATSolveGuided	true	13 439	50 791
	SATSolveGuidedProbeConflicts	false	–	–
	SATSolveGuidedForcedTrue	false	–	–
	SATSolveGuidedEVSIDS	true	12 521	46 118
	SATSolveGuidedShort	true	13 723	52 979
	SATSolveGuidedFast	true	11 991	37 213
SYN657-1.p	SATSolve	false	–	–
	SATSolveGuided	true	10 996	60 140
	SATSolveGuidedProbeConflicts	false	–	–
	SATSolveGuidedForcedTrue	false	–	–
	SATSolveGuidedEVSIDS	false	–	–
	SATSolveGuidedShort	false	–	–
	SATSolveGuidedFast	false	–	–

Table 1: Some random problems solved by different SAT-guided strategies, as a representative example

References

[1] Armin Biere et al. "CaDiCaL 2.0". In: *Computer Aided Verification*. Vol. 14681. Lecture Notes in Computer Science. Springer, 2024, pp. 133–152.

[2] Karel Chvalovský and Jan Jakubův. "Characteristic Subsets of TPTP Benchmarks". In: *Proceedings of the 6th Conference on Artificial Intelligence and Theorem Proving (AITP 2021)* (Aussois, France, Sept. 5–10, 2021). Extended abstract in informal proceedings. 2021.

[3] Reinhold Letz and Gernot Stenz. "Model Elimination and Connection Tableau Procedures". In: *Handbook of Automated Reasoning*. Elsevier, 2001, pp. 2015–2114. ISBN: 978-0-444-50813-3.

[4] Jens Otten. "20 Years of leanCoP — an Overview of the Provers". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*. Vol. 3613. CEUR Workshop Proceedings. 2023.

[5] Michael Rawson and Giles Reger. "Eliminating Models During Model Elimination". In: *Automated Reasoning with Analytic Tableaux and Related Methods*. Ed. by Anupam Das and Sara Negri. Vol. 12842. Cham: Springer International Publishing, 2021, pp. 250–265. ISBN: 978-3-030-86058-5 978-3-030-86059-2.

[6] Fredrik Rømming, Jens Otten, and Sean B. Holden. "Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*. Vol. 3613. CEUR Workshop Proceedings. 2023, pp. 107–118.